

Advanced SQL Window Functions

Practice Assignment — FIRST_VALUE • LAST_VALUE • NTH_VALUE • NTILE • CUME_DIST

How to use this assignment

This worksheet starts with basic questions and gradually becomes more difficult. Solve in order. The PDF contains questions only—no solutions.

Functions: FIRST_VALUE(), LAST_VALUE(), NTH_VALUE(), NTILE(), CUME_DIST(), plus PARTITION BY, ORDER BY, ROWS frames, CASE and subqueries.

1. Schema

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_category VARCHAR(50),  
    brand VARCHAR(50),  
    product_name VARCHAR(100),  
    price DECIMAL(10,2)  
);
```

2. Populated Data

ID	Category	Brand	Product Name	Price
1	Laptop	Dell	Inspiron 15	65,000.00
2	Laptop	HP	Pavilion 14	62,000.00
3	Laptop	Lenovo	IdeaPad Slim 5	58,000.00
4	Laptop	Dell	Vostro 15	72,000.00
5	Laptop	HP	Victus 15	85,000.00
6	Mobile	Samsung	Galaxy A55	42,000.00
7	Mobile	Apple	iPhone 15	70,000.00
8	Mobile	OnePlus	OnePlus 13R	45,000.00
9	Mobile	Samsung	Galaxy S24	75,000.00
10	Mobile	Apple	iPhone 15 Pro	125,000.00
11	Tablet	Samsung	Galaxy Tab S9	55,000.00
12	Tablet	Apple	iPad Air	60,000.00
13	Tablet	Lenovo	Tab P12	35,000.00
14	Tablet	Samsung	Galaxy Tab A9	18,000.00
15	Tablet	Apple	iPad 10th Gen	45,000.00
16	Headphones	Sony	WH-1000XM5	30,000.00
17	Headphones	JBL	Tune 770NC	8,500.00
18	Headphones	Boat	Rockerz 550	2,500.00
19	Headphones	Sony	WH-CH720N	12,000.00
20	Headphones	JBL	Live 660NC	15,000.00
21	Monitor	LG	UltraGear 27	28,000.00
22	Monitor	Samsung	Odyssey G5	32,000.00
23	Monitor	Dell	S2721D	25,000.00
24	Monitor	LG	UltraWide 29	35,000.00
25	Monitor	Samsung	ViewFinity S6	40,000.00

Part A — FIRST_VALUE(): Basic → Intermediate

- Q1.** For every product, display the first product alphabetically by product_name within its product_category.
- Q2.** For every product, display the cheapest product_name in its product_category.
- Q3.** For every product, display the cheapest price in its product_category using FIRST_VALUE().
- Q4.** For every product, show the first product when products are ordered from highest price to lowest price within each category.
- Q5.** For Mobile products only, show the cheapest mobile product using FIRST_VALUE(). Filter Mobile before the window function is evaluated.
- Q6.** For every product, calculate the difference between its price and the cheapest price in its category.

Part B — LAST_VALUE(): Frames Matter

Frame reminder: when you need the final row of the whole partition, use ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

- Q7.** For every product, find the most expensive product_name in its category using LAST_VALUE().
- Q8.** For every product, display the highest price in its category using LAST_VALUE().
- Q9.** For every product, calculate how much cheaper it is than the most expensive product in its category.
- Q10.** For every product, find the last product alphabetically within its category using LAST_VALUE().
- Q11.** For every Laptop, show the most expensive laptop. Make sure the complete partition is included in the frame.
- Q12.** Write two LAST_VALUE() queries: one using the default frame and one using ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING. Compare the results.

Part C — NTH_VALUE(): Pick a Specific Position

- Q13.** For every product, find the 2nd cheapest product in its category.
- Q14.** For every product, find the 3rd cheapest product in its category.
- Q15.** For every product, find the 2nd most expensive product in its category using ORDER BY price DESC.
- Q16.** For Mobile products, display the 3rd most expensive mobile product. Filter Mobile before applying the window function.
- Q17.** For every product, calculate the price difference between the current product and the 2nd most expensive product in its category.
- Q18.** For every product, show the 1st, 2nd and 3rd most expensive product names in its category using three NTH_VALUE() expressions.

Part D — NTILE(): Divide Rows into Buckets

Reminder: NTILE(N) divides ordered rows into N approximately equal buckets. It does not mean rank N.

Q19. Divide all products into 3 price buckets using NTILE(3), ordering price from highest to lowest.

Q20. Using the result, label bucket 1 as Expensive, bucket 2 as Mid Range, and bucket 3 as Cheaper using CASE.

Q21. Divide only Mobile products into 3 buckets based on price DESC and label the buckets Expensive Range Phone, Mid Range Phone and Cheaper Range Phone.

Q22. Divide products into 4 price buckets within each product_category using NTILE(4).

Q23. For each product_category, identify products in the top price quartile (bucket 1 with price DESC).

Q24. Use NTILE(5) within each category and return product_name, category, price and bucket.

Part E — CUME_DIST(): Cumulative Position

Reminder: CUME_DIST() returns a value from 0 to 1 and includes tied rows at the current ORDER BY value.

Q25. Calculate CUME_DIST() for all products ordered by price ASC.

Q26. Calculate CUME_DIST() for all products ordered by price DESC. Explain how the interpretation changes.

Q27. For Mobile products, calculate CUME_DIST() by price ASC and display it as a percentage.

Q28. For every product category, calculate CUME_DIST() by price ASC using PARTITION BY.

Q29. Find products whose CUME_DIST() is 0.50 or less within their category. Use a subquery to filter the window result.

Q30. Within each category, label products as Lower Quarter (≤ 0.25), Lower-Middle (> 0.25 and ≤ 0.50), Upper-Middle (> 0.50 and ≤ 0.75), or Upper Quarter (> 0.75).

Part F — Combined Advanced Challenges

Q31. For every product, show the cheapest product, most expensive product, and current product price within its category using `FIRST_VALUE()`, `LAST_VALUE()`, and the current row.

Q32. For every category, display the 2nd most expensive and 2nd cheapest product using `NTH_VALUE()`.

Q33. For every category, use `NTILE(3)` and `CUME_DIST()` based on price DESC. Return both values for every product.

Q34. For every product, calculate cheapest category price, most expensive category price, current price, and the percentage of the category price range represented by the current product.

Q35. Create a report with category, product, price, cheapest product, most expensive product, 2nd most expensive product, price bucket, and cumulative distribution.

Q36. Within each category, identify products that are in `NTILE(3)` bucket 1 and have `CUME_DIST()` ≥ 0.67 when ordered by price DESC.

Q37. For each brand, find the cheapest product, most expensive product and 2nd most expensive product.

Q38. Compare `NTILE(4)` price quartiles with `CUME_DIST()` within each category. Explain differences when ties exist.

Q39. Return only the most expensive product(s) in every category using `LAST_VALUE()` rather than `MAX()`, and handle ties correctly.

Q40. Final Challenge: Build a complete pricing analysis containing category, product, price, cheapest product, most expensive product, 3rd most expensive product, `NTILE(4)` price quartile, and `CUME_DIST()`. Order by category and price DESC.

Quick Revision Sheet

Function	Meaning
FIRST_VALUE()	First value in the window frame
LAST_VALUE()	Last value in the window frame
NTH_VALUE(col,N)	N-th value in the window frame
NTILE(N)	Divide ordered rows into N approximately equal buckets
CUME_DIST()	Cumulative proportion from 0 to 1
PARTITION BY	Defines the group where calculation restarts
ORDER BY	Defines the order used by the window function
ROWS frame	Defines which physical rows are visible to the function

Recommended order: Q1–6 → Q7–12 → Q13–18 → Q19–24 → Q25–30 → Q31–40.

Mental model: PARTITION BY = which group? ORDER BY = what order? Frame = which rows can the function see?